Stack Data Structure

What will be covered

- Concept Of Stack
- Need and Application of Stack
- Types of Stack
- Basic Operations on Stack
- Representation Of Stack using Array
- Push Operation
- Pop Operation.

Concept Of Stack.

- A stack is an Abstract Data Type (ADT), commonly used in most programming languages. It is named stack as it behaves like a real-world stack, for example – a deck of cards or stack of plates, etc.

- Stack has only end ,this feature makes it LIFO data structure. LIFO stands for Last-in-first-out. Here, the element which is placed (inserted or added) last, is accessed first. In stack terminology, insertion operation is called PUSH operation and removal operation is called POP operation.

Need And Application Of Stack

OBJECT TECHNOLOGIES

Stack is used very often in real life, even normal people use applications of Stack in their daily life routines. Here is some example of the stack in real-life.

- Women Bangles: Women wear a bangle one by one and to pull the first one they have to first pull out the last one.
- Books and Clothes: Piled on top of each other is a great example of the stack.
- Floors in a Building: A person is living on a top floor and wants to go outside, he/she first need to land on the ground floor.
- Browsers: Web browsers use the stack to keep track of the history of web sites if you click back then the previous site opens immediately.

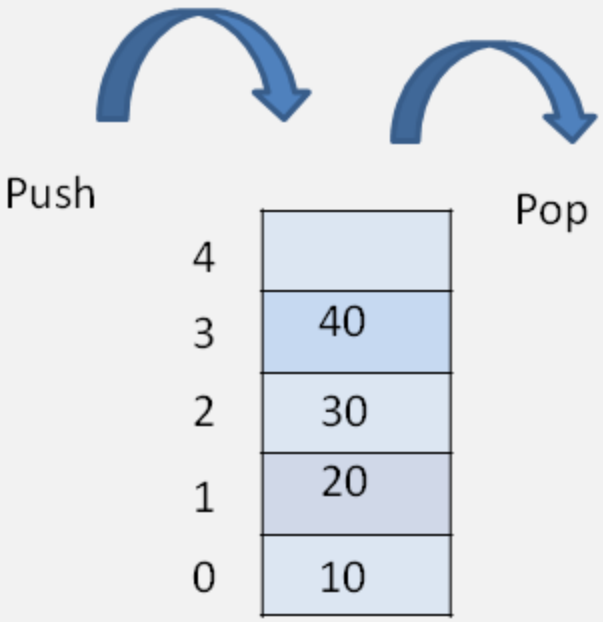
Need And Application Of Stack

- Mobile Phone: Call log in mobiles uses the stack, to get a first-person call log you have to scroll.
- Garage: If a garage is not wide enough. To remove the first car we have to take out all the other cars in after it.
- Text Editors: Undo or Redo mechanism in the Text Editors(Excel, Notepad or WordPad etc.)

Types Of Stack

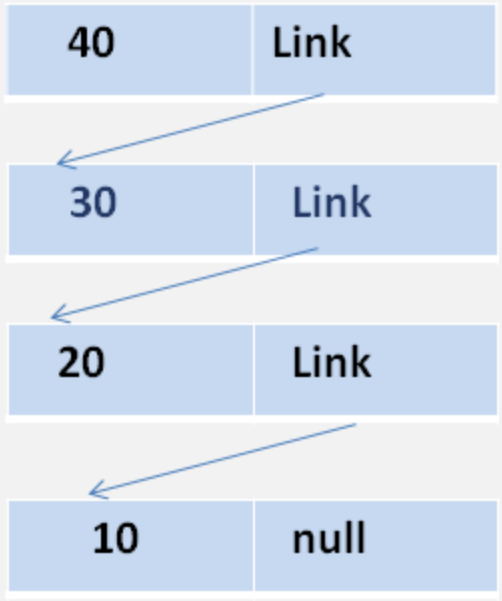
OBJECT TECHNOLOGIES™

1. Linear Stack/Array using Stack



Stack using Array

2. Dynamic Stack



Stack Using Linklist

Basic Operations On Stack

OBJECT TECHNOLOGIES

- **push()** – Pushing (storing) an element on the stack.
- **pop()** – Removing (accessing) an element from the stack.
- When data is PUSHed onto stack or POPPed from stack.

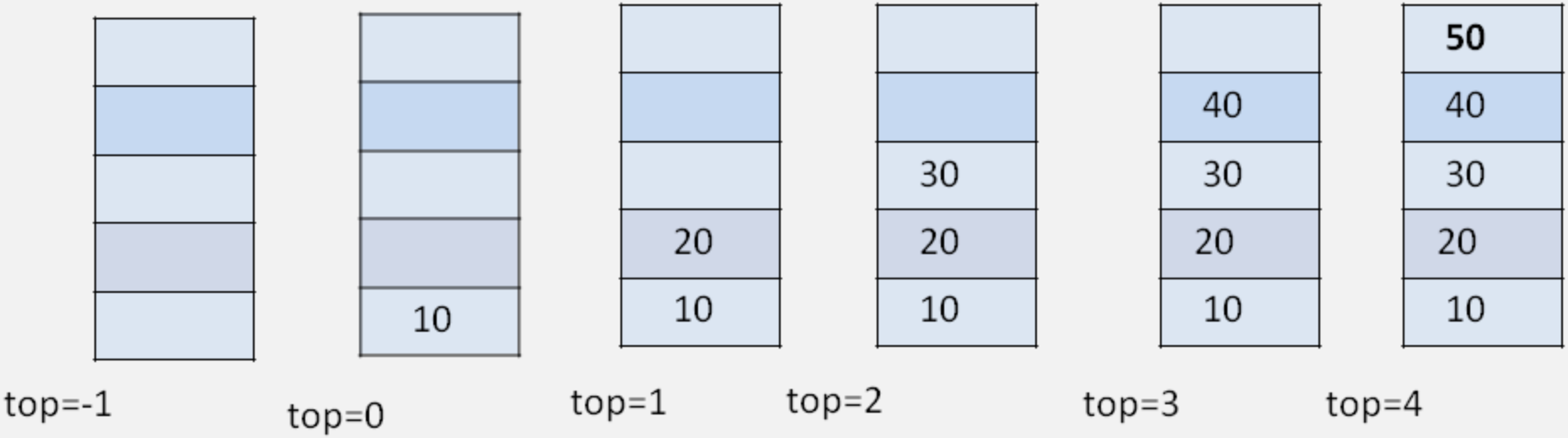
To use a stack efficiently, we need to check the status of stack as well. For the same purpose, the following functionality is added to stacks –

- **peek()** – get the top data element of the stack, without removing it.
- **isFull()** – check if stack is full.
- **isEmpty()** – check if stack is empty.

At all times, we maintain a pointer to the last PUSHed data on the stack. As this pointer always represents the top of the stack, hence named top. The top pointer provides top value of the stack without actually removing it.

Push Operation On Stack.

Step 1 – Checks if the stack is full.
Step 2 – If the stack is full, produces an error and exit.
Step 3 – If the stack is not full, increments top to point next empty space.
Step 4 – Adds data element to the stack location, where top is pointing.
Step 5 – Returns success.



Pop Operation On Stack.

Step 1 – Checks if the stack is empty.
Step 2 – If the stack is empty, produces an error and exit.
Step 3 – If the stack is not empty, accesses the data element at which top is pointing.
Step 4 – Decreases the value of top by 1.
Step 5 – Returns success.